VeOmni

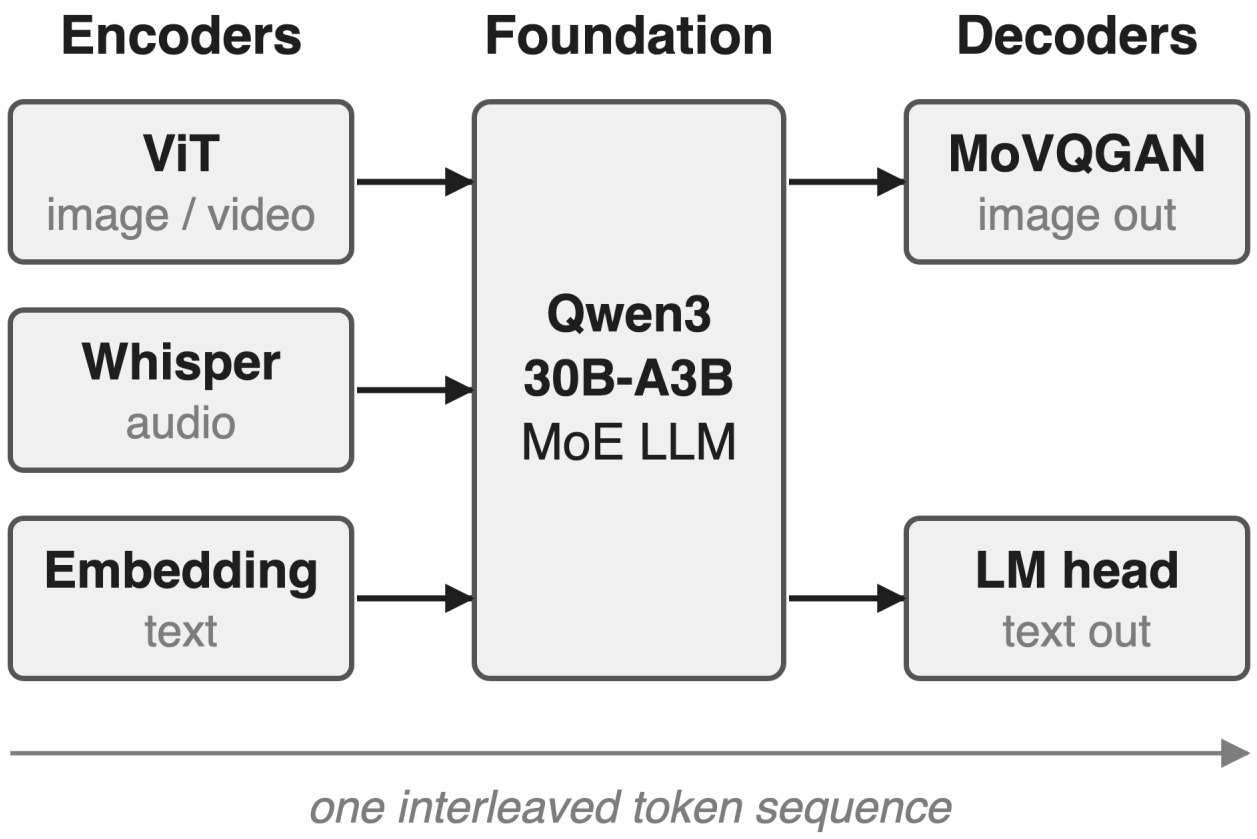
Model-centric distributed recipes for omni-modal training

ByteDance Seed · VeOmni (arXiv 2508.02317, AAAI 2026)

Explainer deck · Industrial Pretraining Architecture & Infra Tech Landscape (2026)

Omni models = encoders + LLM + decoders

- Each part pretrained separately, with its own shape
- One sequence mixes text, image, video, audio
- Goal: train any-to-any, end to end



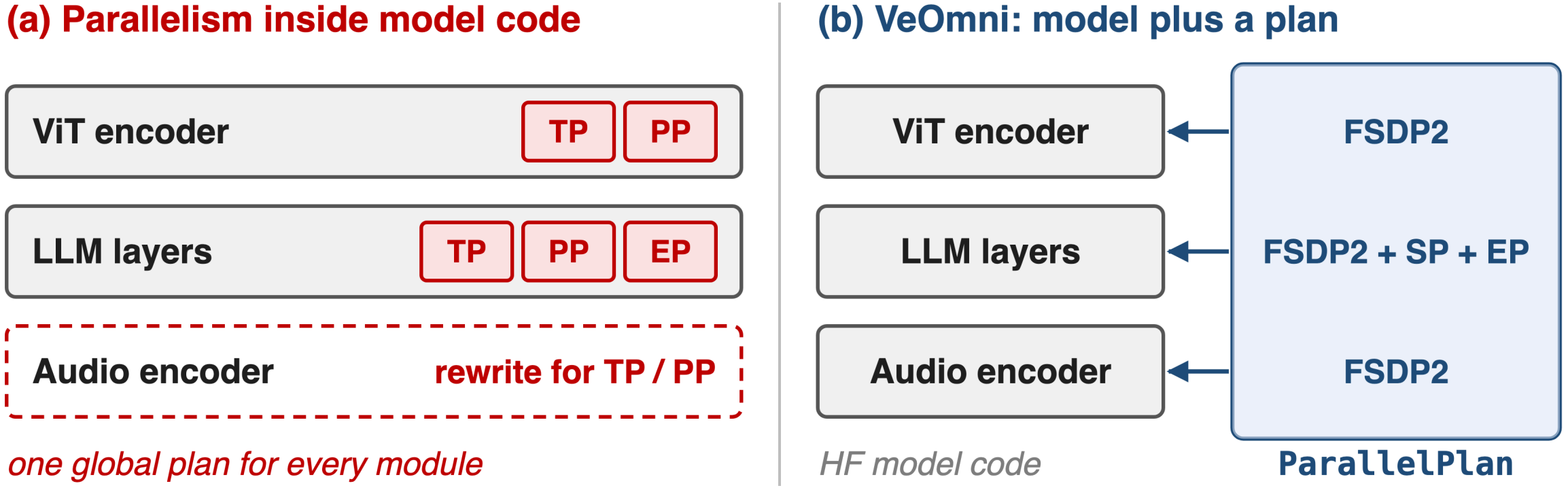
3D parallelism wastes half the GPU cycles

> 48%

of GPU cycles idle: >100B ViT+GPT model, Megatron-LM, 3,000+ GPUs

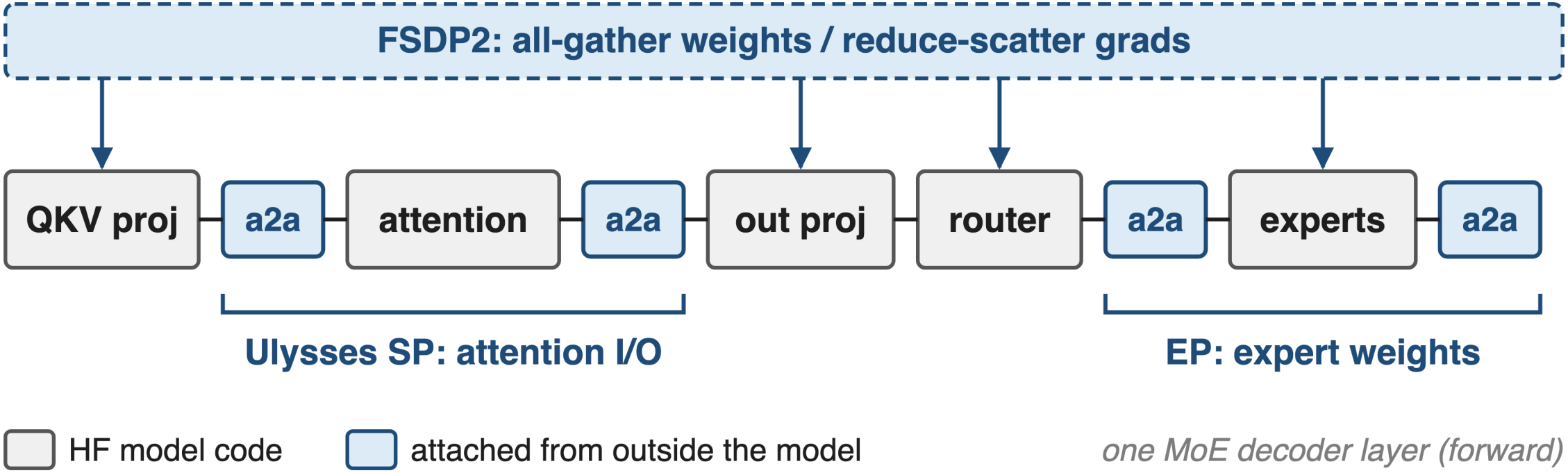
Optimus (ByteDance, ATC'25), arXiv 2408.03505 §2.2; cited by VeOmni as its evidence [17]

In-model parallel code blocks new modalities



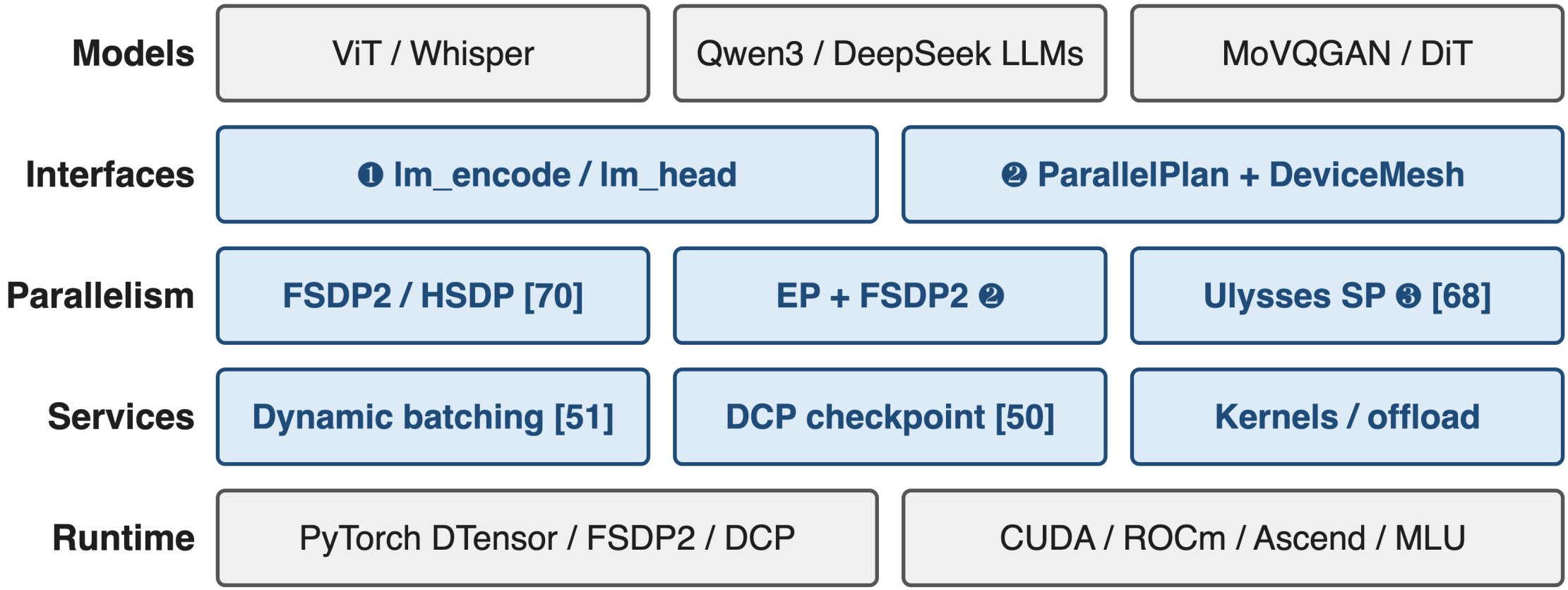
Each new modality means new parallel code, and one global plan fits no module well

Insight: each strategy touches one boundary



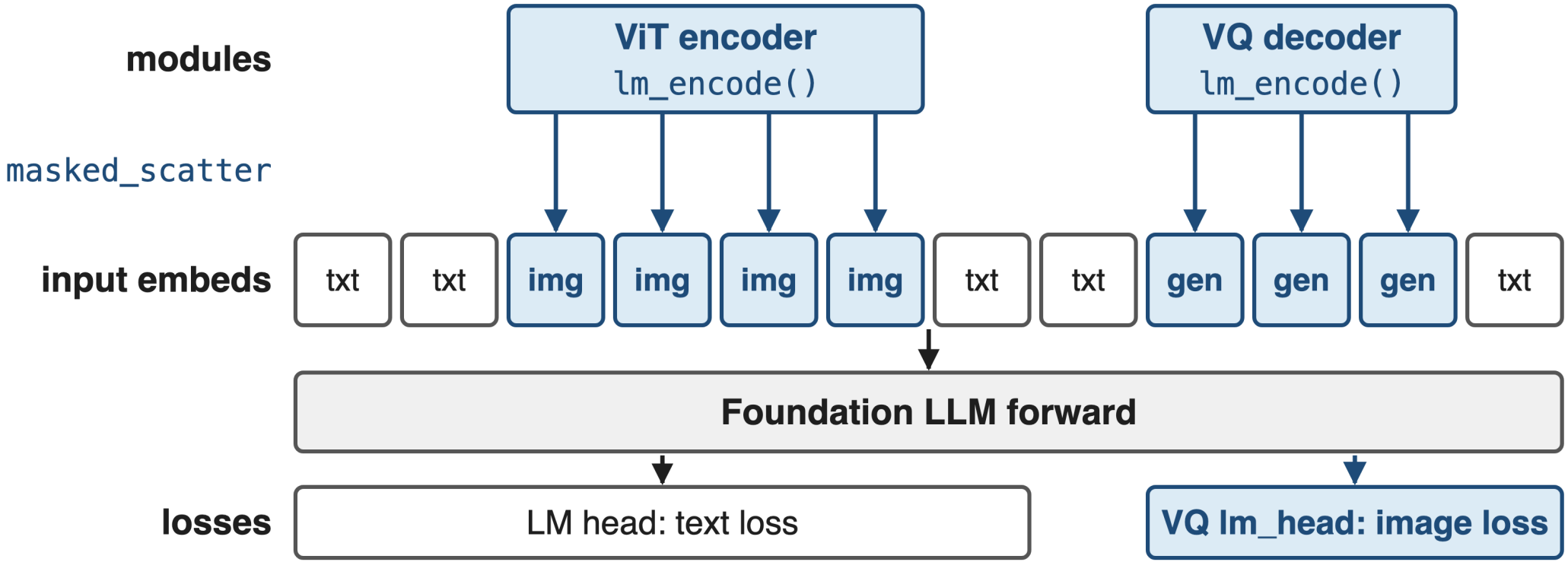
So parallelism can be a per-module plan, applied from outside the model

VeOmni puts recipes under unmodified models



Accent = VeOmni-owned; gray = HF models, PyTorch, hardware; [N] = sibling explainer deck

❶ Modalities plug in via lm_encode and lm_head



Encoders and decoders extend HF PreTrainedModel plus a mixin, with no parallel code

❷ A two-entry plan shards experts on a 2D mesh

qwen3_moe/parallel_plan.py

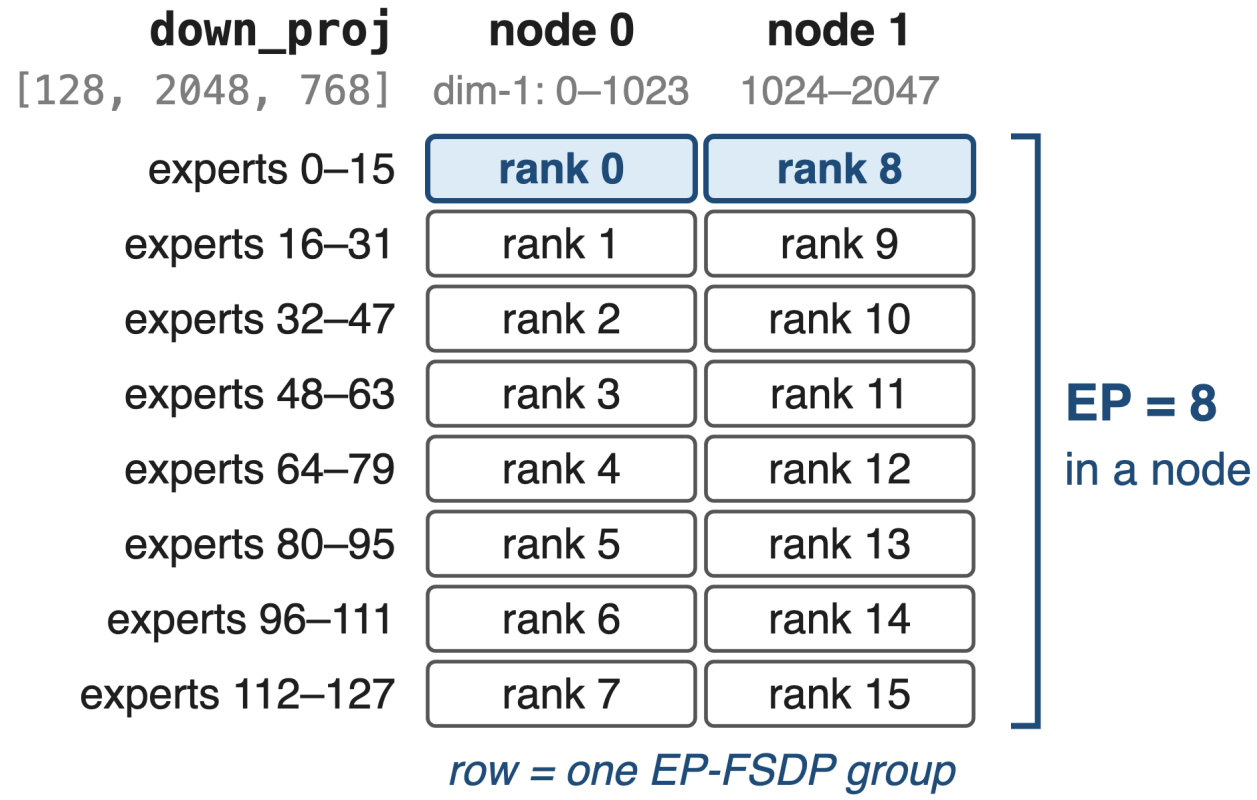
```
ep_plan = {
  "…gate_up_proj": Shard(0),
  "…down_proj":    Shard(0),
}
```

experts: EP 8 on dim-0

+ FSDP2 2-way on dim-1

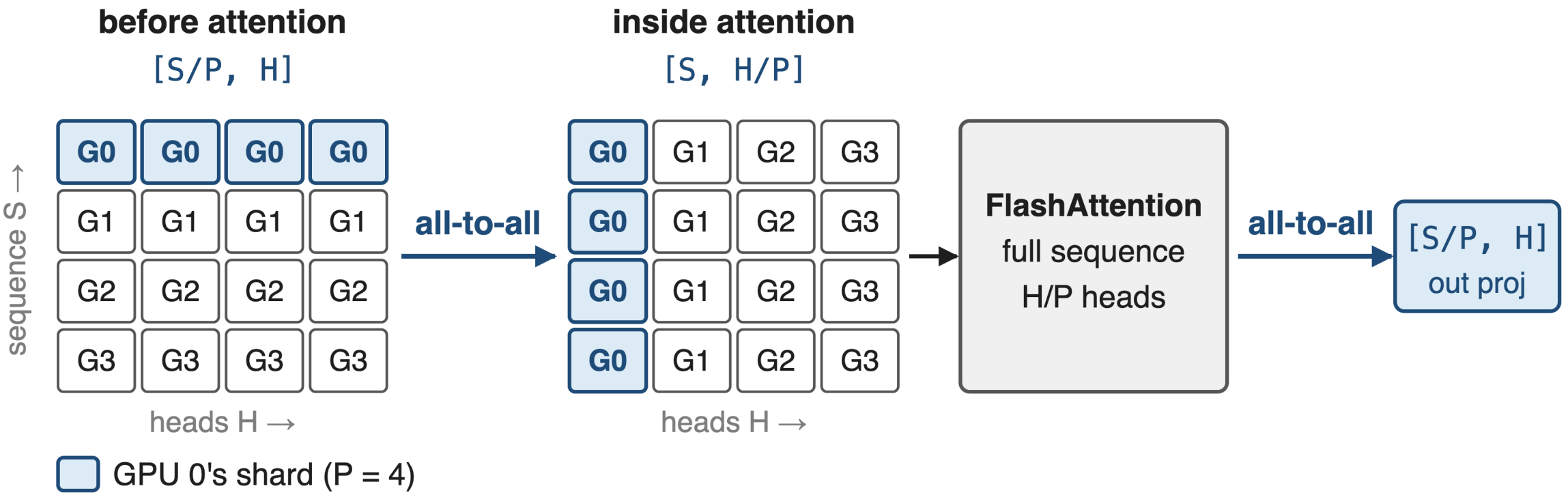
→ `[16, 1024, 768]` per rank

other weights: FSDP2
over all 16 ranks



Worked example from VeOmni's EP+FSDP2 docs: Qwen3-30B-A3B (128 experts) on 2 × 8 GPUs; EP size × EP-FSDP size = world size

❸ Ulysses re-shards attention with two all-to-alls



Only attention sees the full sequence; norms and MLPs keep S/P tokens per GPU

Batching and checkpoints stay outside too

- **Dynamic batching** (deck 51)
- Pack samples to a token budget
- `cu_seqlens` keeps attention per sample

- **DCP checkpoints** (deck 50)
- Weights and optimizer saved as shards
- Reload under a different mesh

Neither service touches the model definition

A 30B omni MoE trains at 2,800+ tokens/s/GPU

2,800+

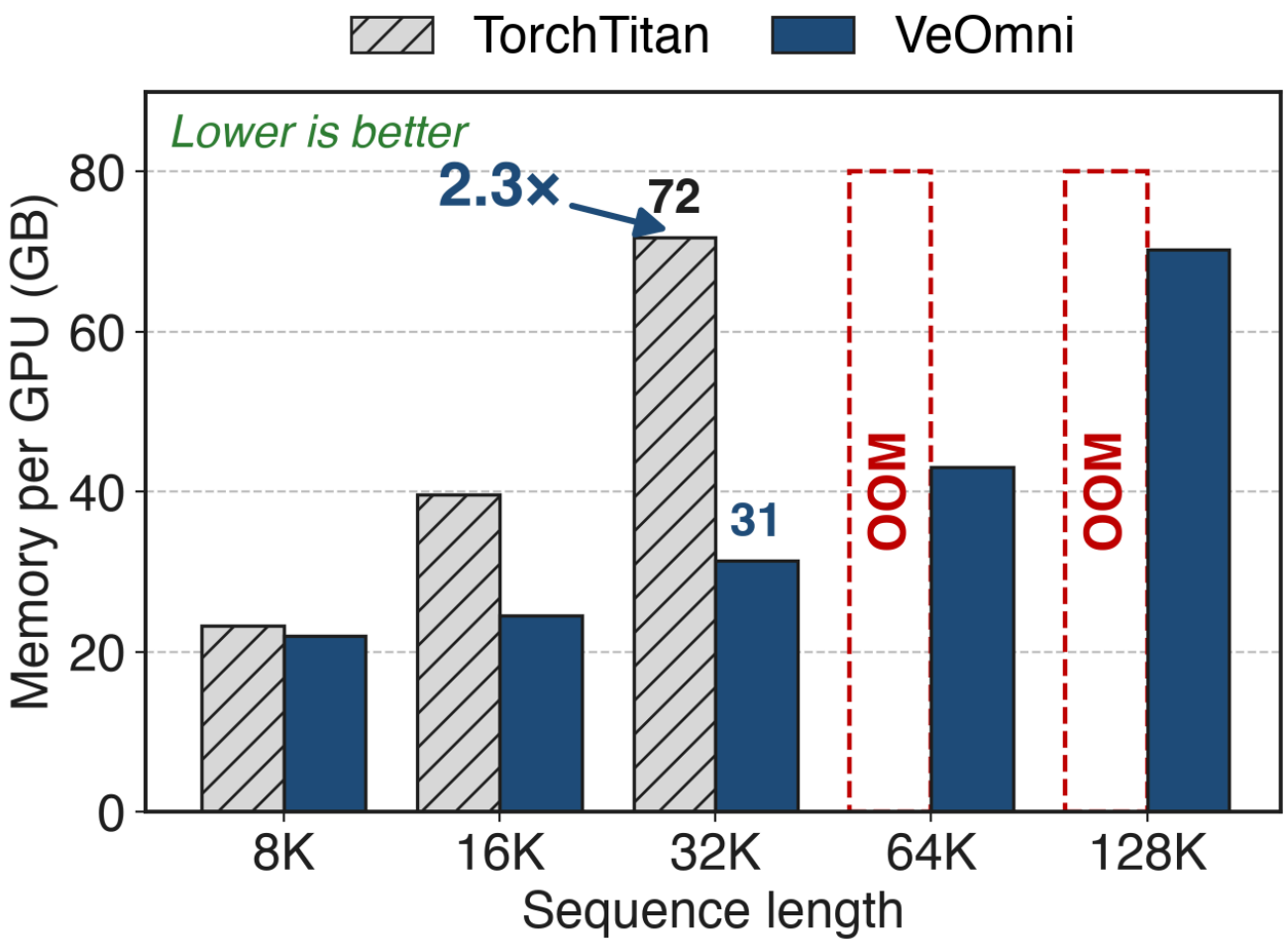
tokens/s/GPU: 30B omni-modal MoE, FSDP + SP + EP on 128 GPUs

- Context reach: **160K** (30B MoE), **192K** at 61.5% MFU (7B), **96K** at 54.82% MFU (72B)

VeOmni arXiv 2508.02317, Abstract and §4.2 (Fig. 4-6)

Same recipe, 2.3× less memory than TorchTitan

- Qwen2-7B text model, FSDP, 128 GPUs
- Both with full activation checkpointing
- TorchTitan OOMs at 64K and 128K



VeOmni spans model families and accelerators

- **Models**: DeepSeek-V3/V4, Qwen3-VL MoE, Qwen3-Omni, Wan 2.1
- **Hardware**: CUDA, ROCm, Ascend, Cambricon MLU
- **Users**: UI-TARS-2, LMMs-Engine, dFactory, LingBot-VLA
- Open since 2025/04; v0.1.0 in 2025/09

The ROCm port adds no HIP code: the training path is device-agnostic

The cost: no PP, per-model patches, low MoE MFU

- **No pipeline parallelism**: listed as future work
- **Per-model files**: omni MoE needs `__init__.py` + 4 more

- Text MoE runs: **7–18%** MFU
- Only text baselines; encoders frozen in eval
- Ulysses: heads divisible by SP size

Decoupled is not free: a plan per model, a patch per architecture

Summary

- Uniform LLM parallelism: **> 48%** GPU cycles idle on a large MLLM
- **Insight**: FSDP, SP, EP each touch one boundary, so plan them per module
- **2,800+** tokens/s/GPU 30B omni MoE; **160K** context; **2.3×** less memory than TorchTitan
- Deep dives: decks 50, 51; related 68, 70, 11



https://arxiv.org/abs/2508.02317